

Кросплатформне програмування

11

Багатопотокове
програмування



Потоки

Про що йтиметься:

- Знайомство з потоками
- Способи створення дочірніх потоків
- Явна реалізація інтерфейсу Runnable
- Створення потоку за допомогою анонімного класу
- Створення потоку за допомогою лямбда-виразу
- Спадкування класу Thread
- Робота з потоками



Знайомство з потоками

У Java є вбудована підтримка багатопоточного програмування. При багатопоточному програмуванні кілька блоків програмного коду виконуються одночасно. Такі одночасної виконувані блоки програмного коду називаються потоками

З одним потоком ми вже неявно мали справу - це головний потік програми, який фактично і є сама програма. З головного потоку можна запускати інші потоки, які називають дочірніми

Для запуску дочірнього потоку нам необхідно :

- визначити програмний код, який повинен виконуватися в рамках потоку, що запускається (дочірнього)
- запустити програмний код у режимі потоку



Знайомство з потоками

Для визначення програмного коду (для виконання в дочірньому потоці) необхідно створити об'єкт класу, що реалізує інтерфейс `Runnable`:

- В інтерфейсі `Runnable` оголошено метод `run()`
- При створенні об'єкта на основі класу, що реалізує інтерфейс `Runnable` визначається код методу `run()`
- Код методу `run()` виконується під час виконання потоку
- Для запуску дочірнього потоку необхідно створити об'єкт класу `Thread` і викликати з цього об'єкта метод `start()`
- Необхідно зв'язати об'єкт, у методі `run()` якого реалізовано програмний код потоку, з об'єктом класу `Thread`, з якого викликається метод `start()` для запуску потоку
- Таке "зв'язування" відбувається при створенні об'єкта класу `Thread`: аргументом конструктору класу `Thread` передається об'єкт з методом `run()`



Знайомство з потоками

Клас `Thread` реалізує інтерфейс `Runnable`, але тільки з порожнім тілом для методу `run()`: у класі `Thread` метод `run()` визначено так, що в ньому немає команд

Якщо створити підклас шляхом успадкування класу `Thread`, то в цьому підкласі можна переозначити метод `run()` (і відповідно, клас реалізує інтерфейс `Runnable`), у цього класу буде метод `start()`, який можна використовувати для запуску дочірнього потоку

Тому один із шляхів створення дочірнього потоку базується на створенні підкласу класу `Thread`



Явна реалізація інтерфейсу Runnable

```
class MyThread implements Runnable{ // Клас реалізує інтерфейс Runnable
    public void run(){ // Опис методу run() ( програмний код для потоку)
        for(int i=1;i<=5;i++){
            System.out.println("Дочірній потік:\t"+i);
            try{
                Thread.sleep(1200); // Затримка у виконанні потоку
            }catch(InterruptedException e){// Обробка винятку
                System.out.println("Переривання дочірнього потоку");
            }
        }
    }
}

class Demo{ // Головний клас
    public static void main(String[] args){ // Головний потік
        System.out.println("Починається виконання програми");
        Thread t=new Thread(new MyThread());// Об'єкт для запуску дочірнього потоку
        System.out.println ("Запускається дочірній потік");
        t.start(); // Запуск дочірнього потоку
        for(int k=0;k<=5;k++){
            System.out.println("Головний потік:\t"+(char) ('A'+k)); // Відображення повідомлення
            try{
                Thread.sleep(2000); // Затримка у виконанні головного потоку
            } // Обробка винятку:
            catch( InterruptedException e){
                System.out.println ("Переривання головного потоку");
            }
        }
        System.out.println ("Виконання програми завершено");
    }
}
```

Результат:

Починається виконання програми
Запускається дочірній потік
Дочірній потік: 1
Головний потік: A
Дочірній потік: 2
Головний потік: B
Дочірній потік: 3
Дочірній потік: 4
Головний потік: C
Дочірній потік: 5
Головний потік: D
Головний потік: E
Головний потік: F
Виконання програми завершено

В принципі в методі main() обробку винятку можна було не виконувати. Але тоді довелось би вказувати throws InterruptedException в описанні методу



Створення потоку з використанням анонімного класу

```
class Demo{
    public static void main(String[] args) throws InterruptedException{
        System.out.println("Починається виконання програми");
        // Аргументом конструктору передається анонімний об'єкт анонімного класу:
        Thread t=new Thread(new Runnable (){
            public void run(){
                for(int i=1;i<=5;i++){
                    System.out.println ("Дочірній потік:\t"+i);
                    try{
                        Thread.sleep(4500);
                    }catch(InterruptedException e){
                        System.out.println("Переривання дочірнього потоку");
                    }
                }
            }
        });
        System.out.println ("Запускається дочірній потік");
        t.start();
        for(int k=0;k<=5;k++){
            System.out.println("Головний потік:\t"+(char) ('A'+k));
            Thread.sleep(2000);
        } // Перевірка дочірнього потоку:
        if(t.isAlive()){
            t.join(); // Очікування завершення дочірнього потоку
        }
        System.out.println("Виконання програми завершено");
    }
}
```

Результат:

Починається виконання програми
Запускається дочірній потік
Дочірній потік: 1
Головний потік: A
Головний потік: B
Головний потік: C
Дочірній потік: 2
Головний потік: D
Головний потік: E
Дочірній потік: 3
Головний потік: F
Дочірній потік: 4
Дочірній потік: 5
Виконання програми завершено

Метод join() може викидати контрольований виняток класу InterruptedException



Створення потоку з використанням лямбда-виразу

```
class Demo{
    public static void main(String[] args) throws InterruptedException{
        System.out.println("Починається виконання програми");
        Runnable r=()->{ // Використано лямбда-вираз
            for(int i=1;i<=5;i++){
                System.out.println("Дочірній потік:\t"+i);
                try{
                    Thread.sleep(4500);
                }catch(InterruptedException e){
                    System.out.println("Переривання дочірнього потоку");
                }
            }
        }; // Інтерфейсна змінна - аргумент конструктора класу Thread:
        Thread t=new Thread(r);
        System.out.println("Запускається дочірній потік");
        t.start();
        for(int k=0;k<=5;k++){
            System.out.println("Головний потік:\t"+(char) ('A'+k));
            Thread.sleep(2000);
        }
        if(t.isAlive()){
            t.join();
        }
        System.out.println("Виконання програми завершено");
    }
}
```

Результат:

```
Починається виконання програми
Запускається дочірній потік
Головний потік:      A
Дочірній потік:      1
Головний потік:      B
Дочірній потік:      2
Головний потік:      C
Дочірній потік:      3
Головний потік:      D
Дочірній потік:      4
Головний потік:      E
Дочірній потік:      5
Головний потік:      F
Дочірній потік:      4
Дочірній потік:      5
Виконання програми завершено
```




Спадкування класу Thread

```
class MyThread extends Thread{           // Підклас класу Thread
    public void run(){                     // Переозначення методу run()
        for(int i=1;i<=5;i++){
            System.out.println("Дочірній потік:\t"+i);
            try{
                Thread.sleep(4500);
            }catch(InterruptedException e){
                System.out.println("Переривання дочірнього потоку");
            }
        }
    }
} // Головний клас:
class Demo{
    public static void main(String[] args) throws InterruptedException{
        System.out.println("Починається виконання програми");
        MyThread t=new MyThread(); // Створення об'єкта класу MyThread
        System.out.println("Запускається дочірній потік");
        t.start(); // Запуск дочірнього потоку
        for(int k=0;k<=5;k++){
            System.out.println("Головний потік:\t"+(char) ('A'+k));
            Thread.sleep(2000);
        }
        if(t.isAlive()){
            t.join();
        }
        System.out.println("Виконання програми завершено");
    }
}
```

Результат – як і в попередньому прикладі



ГОЛОВНИЙ ПОТІК

```
class Demo{
    public static void main(String[] args){
        // Об'єктна змінна для запису посилання на потік:
        Thread t;
        // Отримання посилання на об'єкт головного потоку:
        t=Thread.currentThread();
        // Відображення інформації про потік:
        System.out.println(t);
        // Зміна імені потоку:
        t.setName("Головний потік");
        // Задається пріоритетність потоку:
        t.setPriority(7);
        // Відображення інформації про потік:
        System.out.println(t);
    }
}
```

Результат

Thread[main,5,main]

Thread[Головний потік,7,main]



Створення декількох потоків

```
import java.util.Random;    // Імпорт класу Random
// Підклас MyThread створюється успадкуванням суперкласу Thread:
class MyThread extends Thread{
    private int count;        // Кількість повідомлень
    MyThread(String name,int count){ // Конструктор
        super(name);        // Виклик конструктора суперкласу
        this.count=count;    // Значення цілого поля
        start();             // Запуск потоку виконання
    } // Переозначення методу run():
    public void run(){
        // Відображення повідомлення про запуск потоку:
        System.out.println("Виконується потік "+getName());
        // Створення об'єкта класу Random для створення випадкових чисел:
        Random rnd=new Random();
        // Оператор циклу, в якому виводяться повідомлення:
        for(int k=1;k<=count;k++){
            System.out.println("Повідомлення від потоку "+getName()+" : \t"+getName().charAt(0)+k);
            try{
                // Затримка у виконанні потоку:
                Thread.sleep(1000+rnd.nextInt(2001));
            }
            catch(InterruptedException e){
                System.out.println("Переривання потоку "+getName());
            }
        }
        // Повідомлення про завершення виконання потоку:
        System.out.println("Потік "+getName()+" завершено");
    }
}
// Продовження на наступному слайді!
```



Створення декількох потоків

```
// Головний клас:
class Demo{
    public static void main(String[] args) throws InterruptedException{
        System.out.println("Починає виконуватися головний потік");
        // Створення об'єктів - запуск потоків:
        MyThread A=new MyThread("ALPHA",5);
        MyThread B=new MyThread("BRAVO",3);
        MyThread C=new MyThread("CHARLIE",7);
        // Повідомлення головного потоку:
        for(int k=1;k<=4;k++){
            System.out.println("Повідомлення від головного потоку:\t"+k);
            // Затримка у виконанні потоку:
            Thread.sleep(2000);
        } // Очікування завершення дочірніх потоків:
        if(A.isAlive()){
            A.join();
        }
        if(B.isAlive()){
            B.join();
        }
        if(C.isAlive()){
            C.join();
        } // Повідомлення про завершення головного потоку:
        System.out.println("Головний потік завершено");
    }
}
```

Конструктор класу Thread має декілька версій. Ось деякі:

- без аргументів - створюється об'єкт з параметрами за замовчуванням
- з текстовим аргументом - цей аргумент задає назву потоку
- з аргументом, що є об'єктом класу, який реалізує інтерфейс Runnable - виконується метод run() відповідного об'єкта
- перший аргумент - об'єкт з методом run(), другий текстовий аргумент - назва потоку

Результат:

```
Починає виконуватися головний потік
Виконується потік CHARLIE
Виконується потік ALPHA
Виконується потік BRAVO
Повідомлення від головного потоку: 1
Повідомлення від потоку ALPHA:    A1
Повідомлення від потоку CHARLIE:   C1
Повідомлення від потоку BRAVO:     B1
Повідомлення від потоку ALPHA:     A2
Повідомлення від головного потоку: 2
Повідомлення від потоку ALPHA:     A3
Повідомлення від потоку CHARLIE:   C2
Повідомлення від потоку BRAVO:     B2
Повідомлення від головного потоку: 3
Повідомлення від потоку ALPHA:     A4
Повідомлення від потоку BRAVO:     B3
Повідомлення від потоку CHARLIE:   C3
Повідомлення від головного потоку: 4
Повідомлення від потоку CHARLIE:   C4
Повідомлення від потоку ALPHA:     A5
Потік BRAVO завершено
Потік ALPHA завершено
Повідомлення від потоку CHARLIE:   C5
Повідомлення від потоку CHARLIE:   C6
Повідомлення від потоку CHARLIE:   C7
Потік CHARLIE завершено
Головний потік завершено
```



Створення демон-потоків

Є спеціальний тип потоків, які називаються демон-потоками

Зазвичай такі потоки виконуються у фоновому режимі та відіграють деяку "допоміжну" роль щодо "обслуговування" інших потоків

Головна особливість демон-потоків полягає в тому, що при завершенні головного потоку демон-потік завершується автоматично



Програма: створення демон-поток

Ми розглянемо задачу про обчислення суми натуральних чисел

Програму організуємо із залученням дочірнього демон-поток: під час запуску програми запускається демон-потік, у якому виконується обчислення суми натуральних чисел

Кожне нове значення для суми відображається у консольному вікні. Одночасно з головного поток відображається діалогове вікно, в якому користувач повинен натиснути одну з двох кнопок (Yes або No)

Якщо користувач натискає кнопку No, вікно закривається, але процес обчислення суми продовжується, і вікно з двома кнопками з'явиться знову. Все це триває доти, доки користувач не натисне у вікні кнопку Yes



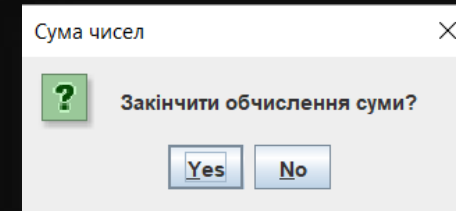
Програма: створення демон-поток

```
// Статичний імпорт:
import static javax.swing.JOptionPane.*;
class Demo{
    public static void main(String[] args) throws InterruptedException{
        // Створення об'єкта дочірнього потоку. Перший
        // аргумент конструктора є лямбда-виразом,
        // другий аргумент - назва потоку:
        Thread t=new Thread(()->{
            // Індексна змінна та
            // змінна для запису суми чисел:
            int k=1,s=0;
            // Нескінченний цикл для обчислення суми:
            while(true){
                // Відображення повідомлення:
                System.out.println(Thread.currentThread().getName()+" : "+s);
                try{
                    // Затримка у виконанні потоку:
                    Thread.sleep(1000);
                } // Обробка винятку:
                catch(InterruptedException e){}
                // Доданок до суми:
                s+=k;
                // Збільшення значення індексної змінної:
                k++;
            }
        },"Обчислення суми"); // Продовження на наступному слайді!
```



Програма: створення демон-поток

```
// Статус демон-поток:  
t.setDaemon(true);  
// Змінна для запису результату  
// вибору користувача (натиснена кнопка):  
int res;  
// Запуск поток на виконання:  
t.start();  
// Відображення діалогового вікна:  
do{  
    // Затримка у виконанні поток:  
    Thread.sleep(3000);  
    // Відображення вікна та запам'ятовування  
    // вибору користувача:  
    res=showConfirmDialog(null,  
        // Текст у вікні:  
        "Закінчити обчислення суми?",  
        // Назва вікна:  
        "Сума чисел",  
        // Кнопки у вікні:  
        YES_NO_OPTION  
    );  
}while(res!=YES_OPTION);  
}
```



Результат:

Обчислення суми: 0
Обчислення суми: 1
Обчислення суми: 3
Обчислення суми: 6
Обчислення суми: 10
Обчислення суми: 15
Обчислення суми: 21
Обчислення суми: 28
Обчислення суми: 36
Обчислення суми: 45
Обчислення суми: 55

Можуть використовуватися такі константи для визначення кнопок у діалоговому вікні:
DEFAULT_OPTION (за замовчуванням кнопка OK)
YES_NO_OPTION (Кнопки Yes та No)
YES_NO_CANCEL_OPTION (Кнопки Yes, No та Cancel)
OK_CANCEL_OPTION (Кнопки OK та Cancel)

Результат методу showConfirmDialog():
YES_OPTION (Кнопка Yes)
NO_OPTION (Кнопка No)
CANCEL_OPTION (Кнопка Cancel)
OK_OPTION (Кнопка OK)
CLOSED_OPTION (Системна піктограма)



Синхронізація потоків

Нерідко трапляється так, що кілька потоків у процесі виконання звертаються до загального ресурсу. Тут може виникнути проблема, пов'язана з тим, що кілька потоків намагаються одночасно виконати деякі операції з цим загальним ресурсом

Вирішення проблеми може бути пов'язане із синхронізацією потоків при зверненні до ресурсу

Ідея синхронізації проста: загальний ресурс при зверненні до нього одного потоку блокується для інших потоків

Якщо йдеться про синхронізацію потоків при зверненні до деякого об'єкта, використовується такий шаблон:

```
synchronized(об'єкт) {  
    // код синхронізації  
}
```

Якщо потрібно синхронізувати метод (такий метод буде доступний у кожний момент лише для одного потоку), то в сигнатурі методу вказується ключове слово **synchronized**



Синхронізація потоків

```
class MyNumber{                                // Клас із цілим полем
    int number;
} // Клас створення потоків:
class MyThread extends Thread{
    private MyNumber obj;                      // Посилання на об'єкт із цілим полем
    private int time;                          // Довжина паузи у виконанні потоку
    private int count;                        // Кількість циклів
    private boolean state;                    // Поле логічного типу
    MyThread (String name, MyNumber obj, int time, int count, boolean state){
        super(name);                          // Виклик конструктора суперкласу
        this.obj=obj;                         // Посилання на об'єкт
        this.time=time;                      // Інтервал затримки
        this.count=count;                   // Кількість циклів
        this.state=state;                   // Значення поля логічного типу
        start();                             // Запуск потоку
    } // Метод для відображення "лінії" із символів:
    private void line(){
        char s;                              // Локальна символна змінна
        if(state) s='-';                     // Значення змінної
        else s='*';
        for(int k=1;k<=35;k++){              // Відображення послідовності символів
            System.out.print(s);
        }
        System.out.println();
    }
}
// Продовження на наступному слайді!
```



Синхронізація потоків

```
// Перевизначення методу run():
public void run(){
    for(int k=1;k<=count;k++){          // Оператор циклу
        synchronized(obj){             // Блок синхронізації
            line();                      // Відображається "лінія"
            // Відображення повідомлення:
            System.out.println("Потік "+getName()+": вихідне значення "+obj.number);
            try{                         // Контрольований код
                Thread.sleep(time);      // Затримка у виконанні потоку
            } // Обробка винятку:
            catch(InterruptedException e){
                System.out.println(e);
            } // Зміна значення поля об'єкта:
            if(state) obj.number++;
            else obj.number--;
            // Відображення повідомлення:
            System.out.println("Потік "+getName()+": нове значення "+obj.number);
            line();                      // Відображення "лінії"
        } // Завершення блоку синхронізації
    }
}
// Продовження на наступному слайді!
```



Синхронізація потоків

```
// Головний клас:
class Demo{
    // Головний метод:
    public static void main(String[] args){
        // Цілочислові змінні:
        int n=100,count=5,time=1000,dt=200;
        // Створення об'єкта з цілим полем:
        MyNumber obj=new MyNumber();
        // Надає полю об'єкта значення:
        obj.number=n;
        // Створення першого потоку:
        MyThread Alpha=new MyThread("Alpha",obj,time+dt,count,true );
        // Створення другого потоку:
        MyThread Bravo=new MyThread("Bravo",obj,time-dt,count,false);
        // Контрольований код:
        try{
            // Очікування завершення потоків:
            if(Alpha.isAlive()) Alpha.join();
            if(Bravo.isAlive()) Bravo.join();
        } // Обробка виключення:
        catch (InterruptedException e){
            System.out.println(e);
        }
    }
}
```

Синхронізація потоків

Результат:

```
-----
Потік Alpha: вихідне значення 100
Потік Alpha: нове значення 101
-----
*****
Потік Bravo: вихідне значення 101
Потік Bravo: нове значення 100
*****
-----
Потік Alpha: вихідне значення 100
Потік Alpha: нове значення 101
-----
-----
Потік Alpha: вихідне значення 101
Потік Alpha: нове значення 102
-----
-----
Потік Alpha: вихідне значення 102
Потік Alpha: нове значення 103
-----
*****
Потік Bravo: вихідне значення 103
Потік Bravo: нове значення 102
*****
-----
Потік Alpha: вихідне значення 102
Потік Alpha: нове значення 103
-----
*****
Потік Bravo: вихідне значення 103
Потік Bravo: нове значення 102
*****
*****
Потік Bravo: вихідне значення 102
Потік Bravo: нове значення 101
*****
*****
Потік Bravo: вихідне значення 101
Потік Bravo: нове значення 100
*****
```



Методи роботи з потоками - 1

Метод	Описання
activeCount()	Методом повертається кількість активних потоків у групі
checkAccess()	Метод дозволяє виконати перевірку щодо того, чи може вказаний потік бути змінений виконуваним потоком
currentThread()	Метод повертає результатом посилання на об'єкт поточного потоку (потіку, у якому викликається метод)
enumerate()	Метод призначений для копіювання в масив посилань на всі активні потоки в цій групі потоків (і її підгрупах)
getId()	Методом як результат повертається ідентифікатор потоку - ціле число, що ідентифікує потік
getName()	Метод як результат повертає ім'я потоку
getPriority()	Метод повертає результатом пріоритет потоку (ціле число в діапазоні значень від 1 до 10)
getState()	Метод повертає об'єкт, що визначає статус потоку
getThreadGroup()	Результатом методу є об'єкт, що визначає групу, до якої належить потік

Методи роботи з потоками - 2

Метод	Описання
holdsLock()	Метод результатом повертає логічне значення, що визначає, чи утримує метод монітор (спеціальний об'єкт, що використовується для блокування деякого ресурсу з метою недопущення звернення до нього відразу кількох методів)
interrupt()	Метод призначений для виконання переривання потоку
interrupted()	Метод використовується для перевірки того, чи потік був перерваний. При цьому змінюється статус потоку (щодо переривання)
isAlive()	Метод для визначення того, чи активний потік
isDaemon()	Перевірка потоку щодо того, чи є він демон-поток (такі потоки зазвичай виконуються у фоновому режимі і автоматично завершуються при завершенні головного потоку)
isInterrupted()	Метод використовується для перевірки того, чи потік був перерваний. При цьому статус потоку (щодо переривання) не змінюється
join()	Виклик методу призводить до того, що потік, у якому викликається метод, очікує завершення потоку, з об'єкта якого викликається метод

Методи роботи з потоками - 3

Метод	Описання
notify()	При виклику методу один із потоків, що знаходяться в режимі очікування (для отримання доступу до ресурсу), переводиться в режим виконання
notifyAll()	При виклику методу всі потоки, що перебувають у режимі очікування (для отримання доступу до ресурсу), переводяться в режим виконання
run()	Метод із кодом, що виконується в рамках потоку. Визначає точку входу в потік
setDaemon()	Метод дозволяє визначити потік як демон-потік (особливий вид потоків, які автоматично завершуються після завершення виконання головного потоку)
setName()	Метод дозволяє встановити назву для потоку
setPriority()	Метод призначений для надання значення пріоритету потоку



Методи роботи з потоками - 4

Метод	Описання
sleep()	Метод дозволяє призупинити виконання потоку. Аргументом методу передається ціле число, що визначає час (у мілісекундах) припинення у виконанні потоку
start()	Метод викликається під час запуску потоку виконання
wait()	Метод призначений для переведення потоку в режим очікування (при отриманні доступу до ресурсу)
yield()	Методом надсилається сигнал диспетчеру потоків, що поточний потік готовий поступитися використанням процесора на користь інших потоків.

Дякую за увагу!

Запитання?